

For this assignment I built a multinomial logistic regression model for news topic classification. The model had to look at news articles from 3 languages, xhosa, English and Shona. For these 3 languages it had to classify which category each one of the articles falls under.

OOP Design

The system is built around a core class, `MultinomialLogisticRegression(nn.Module)`. It implemented 3 functions. `Forward`, which returns raw logits, kept unnormalized so that `nn.CrossEntropyLoss`'s internal softmax is not applied in double. `Compute_probabilities`, which applies softmax to forward's output and `predict`, which returns argmax over those probabilities as the final class prediction.

The rest of the system uses separate functions for each part of the overall model.

`load_language` loads the train, development, and test files and creates a `full_text` column by combining the headline and article body. `clean_text` and `add_clean_column` lowercase the text and remove punctuation.

`build_vectoriser` and `vectorise_splits` create either a `CountVectorizer` or `TfidfVectorizer`. The vectoriser is fitted only on the training data, and the same fitted vocabulary is then used to transform the development and test sets. This prevents information from the development or test sets from leaking into training.

`train_model` performs mini-batch stochastic gradient descent and can stop early when validation accuracy stops improving. `evaluate` calculates accuracy, micro and macro precision, recall and F1, as well as a full classification report. Finally, `run_training` combines these functions into one reusable training route.

Reproducibility is enforced by calling a `set_seed()` function as the first line of every training function, ensuring identical results across repeated runs of the same configuration checked by rerunning a specific hyperparameter configuration twice and getting the same output.

Data & Preprocessing

MasakhaNEWS provides pre-split train/dev/test sets for each language, with a distinct class set per language: English has 3309 train examples, 6 classes: business, entertainment, health, politics, sports, technology and roughly balanced, 429–700 per class. IsiXhosa has 1032 train examples, 5 classes, missing technology and is imbalanced, ranging from 50 business examples to 350 entertainment examples. chiShona has 1 288 train examples, 4 classes: politics, business, health, sports and its decently balanced 291–350 per class.

Text cleaning applied lowercasing and punctuation stripping deliberately keeping diacritics rather than stripping them, since these carry meaningful differences in isiXhosa and chiShona.

Hyperparameter Tuning Process

Given five hyperparameters to be tuned, feature extraction method, vocabulary size, minimum frequency, learning rate and batch size a full on iterative loop that went through all

optimal hyperparams didn't make sense computationally. Instead, a two stage approach was used. Stage one fixed feature settings (TF-IDF, 5 000 features, min-frequency 3) and swept learning rate (0.01–1.0) with batch size (16/32/64) to identify a good optimizer configuration. Then stage two dealt with the rest. It fixed that winning learning rate and batch size and looking for feature-extraction method (BoW vs. TF-IDF) with vocabulary size (2000/5000/10000) and minimum frequency (0/3/5/10).

Across all three languages, batch size 16 with the exception of chiShona at 32 with learning rate 1.0 outperformed larger batches and smaller learning rates. A failure mode emerged at low learning rate combined with large batch size as with fewer gradient updates per epoch, patience based early stopping with monitoring validation accuracy would trigger prematurely, mistaking genuinely slow convergence for a plateau. For isiXhosa, a learning rate of 1.0 and batch size 16 produced the strongest development during the hyper param tuning, reaching 87.1% accuracy and a macro F1 of 0.655. Larger batch sizes performed worse, particularly at the highest learning rate. These settings were therefore fixed for the subsequent feature extraction experiments.

Feature-extraction method preference diverged sharply by language: TF-IDF outperformed BoW for English (88.8% dev accuracy) and chiShona (90.8%), both balanced datasets, while BoW outperformed TF-IDF for isiXhosa (91.2% vs 88.4%) the only imbalanced dataset of the three. This shows TF-IDF's suppressing influence from words that do not repeat often enough within an under represented class with few examples, whereas raw counts preserve that signal undiminished. The fact that chiShona, equally small but balanced still favoured TF-IDF suggests imbalance, not dataset size alone, drives this result.

An early version of the training pipeline did not call `set_seed()` inside each experiment function, only once at the top of the notebook. This caused irreproducible results across reruns of nominally identical configurations most visibly, one BoW configuration appeared to collapse to 43% accuracy in one run, but reached a normal 86% when rerun. Moving the seed reset to the start of every model-creating function eliminated this. Check out “ran a bunch of tests to figure out what's happening with bow but i just found that my seed wasn't being ran before running my experiments” to see the update in my gitrepo

Results

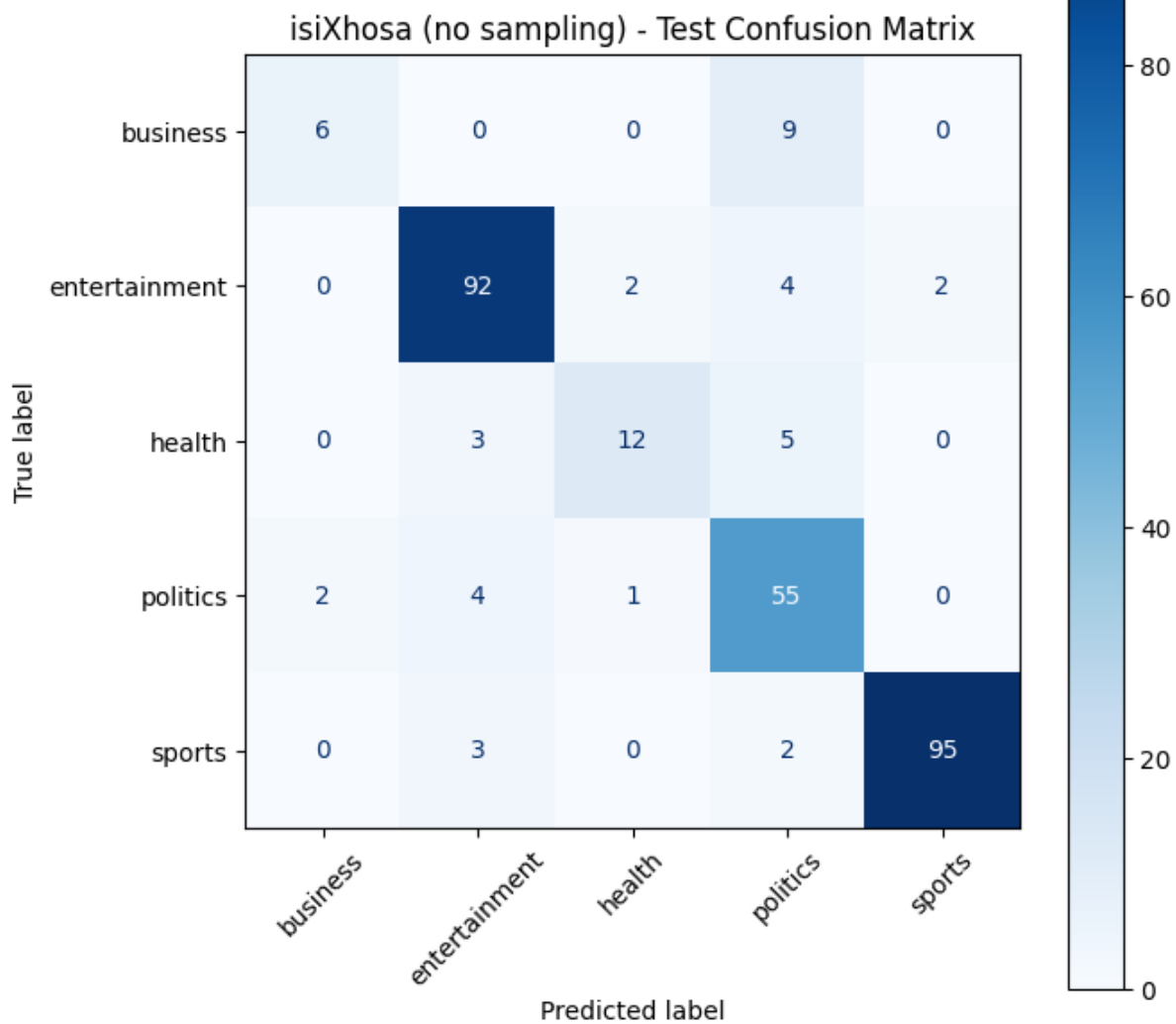
Table 1: Best configuration per language (test set)

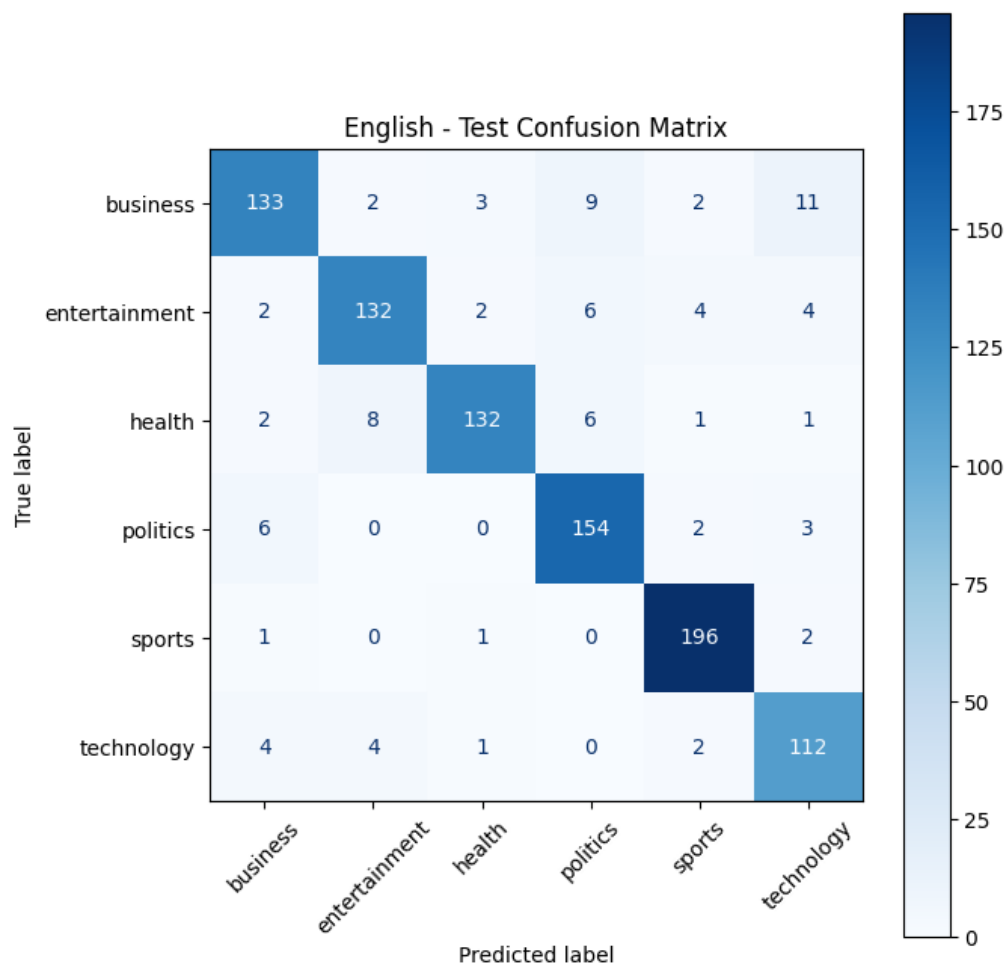
Language Best configuration		Test accuracy	Test macro-F1
English	TF-IDF, 8,434 vocab (min_freq=10), lr=1.0, bs=16	0.906	0.902
isiXhosa	BoW, 10,000 vocab (min_freq=1), lr=1.0, bs=16	0.875	0.777

chiShona	TF-IDF, 5,000 vocab (min_freq=1), lr=1.0, bs=32	0.921	0.925
----------	--	-------	-------

Table 2:

Approach	Test accuracy	Test macro-F1
No sampling (baseline)	0.875	0.777
Upsampling (boost factor = 1.0)	0.902	0.818
Downsampling	best dev config, ratio=3.0, macro-F1=0.775, underperformed baseline and not evaluated on test	
Bilingual: isiXhosa + chiShona	0.902	0.833
Bilingual: isiXhosa + English	0.855	0.634





Weight Analysis (English)

Inspecting the trained linear layer's weights for the business class showed the highest weighted words were business, prices, energy, bank, workers, customers, shares, firm, oil which is plausible business vocabulary with no evidence of overfitting.

The technology class told a more nuanced story. Alongside genuine topical terms (cyber, data, digital, robot, online), its top weights included other names and words like musk, tiktok, google, Microsoft. A model relying heavily on brand names risks failing on genuinely tech related articles that don't mention these particular entities.

Class Imbalance & Bilingual Training

Sampling strategies. Upsampling via `WeightedRandomSampler`, inversely weighting by class frequency improved isiXhosa's macro-F1 from 0.777 to 0.818, with the clearest gains on the two minority classes: business F1 rose from 0.52 to 0.58, health from 0.69 to 0.76, without materially harming majority-class performance. Downsampling, by contrast, underperformed the nonsampling baseline at every tested ratio, with performance improving as the downsampling increased its target ratio, consistent with isiXhosa's small dataset (1 032 train rows) being unable to afford discarding real majority class examples in exchange for balance.

Bilingual training.

isiXhosa+chiShona achieved the best isiXhosa result of the entire study (test macro-F1 0.833), while isiXhosa+English initially performed the worst of any strategy tested (0.634). Inspecting training curves revealed the isiXhosa+English run was numerically unstable at the learning rate carried over unchanged from isiXhosa's solo tuning train loss starting near 137, with great fluctuation. Rerunning with a lower learning rate 0.1 substantially recovered performance macro-F1 0.798. Even after this correction, however, isiXhosa+chiShona still outperformed the tuned isiXhosa+English result, suggesting genuine linguistic relation between the two Bantu languages provided real benefit beyond what class overlap or data volume alone would predict.

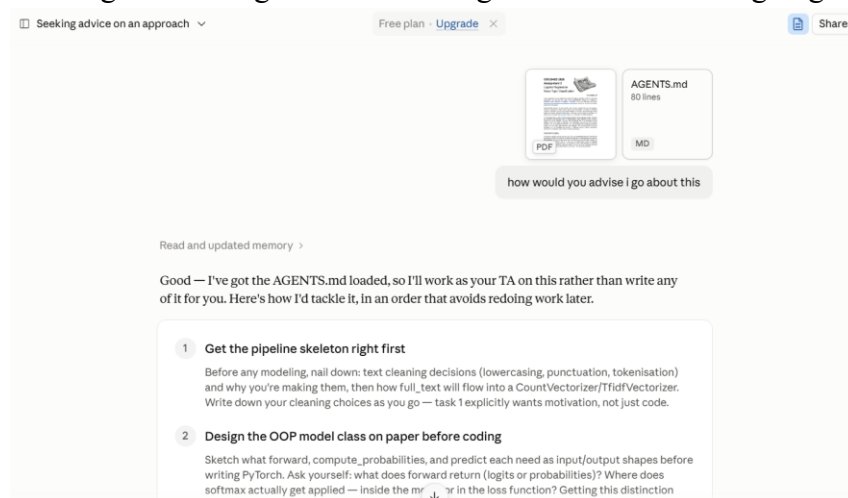
Micro vs. Macro & Confusion Matrix Analysis

Across the three languages, micro and macro metrics diverged only for isiXhosa with a baseline accuracy 0.875 versus macro-F1 0.777, a 10-point gap, compared to English (0.906 vs. 0.902) and chiShona (0.921 vs. 0.925).

The confusion matrix for isiXhosa's baseline model showed business's dominant error was systematic confusion with politics with 9 of 15 true business articles misclassified as politics, possibly reflecting genuine overlap between the two. This specific confusion shrank consistently across every improvement strategy: 9 misclassified under baseline, 8 under upsampling, 5 under isiXhosa+chiShona bilingual training directly demonstrating that bilingual training provided the largest improvement for isiXhosa's rarest class, though health 10–20 test examples remained comparably weak across every strategy tested, suggesting these interventions do not uniformly help all minority classes.

AI Usage

I used Claude during the development of this assignment primarily for how I should go about thinking of the assignment and things to consider before going in head first.



For the hyperparameter search, I ask how to efficiently structure loops over different combinations of parameters. I already had an approach in mind, and the discussion mainly

i need an efficient way to run combinations of hyperparameters using a loop, what's the best way to approach this?

Explaining hyperparameter grid search design at a high level.

A few design decisions here matter more than the loop mechanics themselves, since your hyperparameters aren't all equally "cheap" to vary.

Separate the hyperparameters by cost, not just by name

Your grid has: learning rate, batch size, vocab size, min frequency, and (feature type: BoW vs TF-IDF). Notice that vocab size and min frequency change what your *vectorizer* produces — they require refitting `CountVectorizer` / `TfidfVectorizer`. Learning rate and batch size only change how you train on data you already have. If you nest the loop naively (vectorize inside the innermost loop), you'll refit the same vectorizer dozens of times for no reason. Structure it so vectorizer-dependent params are the outer loop, and refit only when those change:

confirmed it.